

Panduan Git Workflow Tim Hexacore

Dokumen ini berisi standar alur kerja (*workflow*) Git yang wajib dipatuhi oleh seluruh anggota tim Hexacore (Backend, Frontend, DevOps, dan QA) dalam pengembangan aplikasi **LenteraPustaka**.

1. Strategi Branching (GitHub Flow)

Mulai tahap ini, kita secara ketat menggunakan **GitHub Flow**. Aturan utamanya adalah:

-  **DILARANG KERAS** melakukan **push** langsung ke *branch main*.
-  *Branch main* adalah *branch* utama yang harus selalu bersih, stabil, dan siap *deploy*.
- Semua pengerjaan fitur, perbaikan *bug*, atau dokumentasi harus dilakukan di *branch* terpisah dan digabungkan ke *main* melalui **Pull Request (PR)**[cite: 1].

2. Aturan Penamaan Branch (Branch Naming)

Gunakan format: **type/deskripsi-singkat** (huruf kecil, gunakan tanda hubung - untuk spasi)[cite: 1].

Tipe	Kapan Digunakan	Contoh
feature/	Menambah fitur baru (UI/API)	feature/dashboard-admin
fix/	Memperbaiki <i>bug</i> atau <i>error</i>	fix/login-auth-error
docs/	Update dokumen/README	docs/api-documentation
refactor/	Merapikan kode tanpa ubah fitur	refactor/split-crud-file
chore/	Setup/konfigurasi (<i>Docker</i> , <i>deps</i>)	chore/update-requirements

3. Konvensi Commit Message

Kita menggunakan **Conventional Commits**[cite: 1]. Formatnya: **type: deskripsi singkat**.

- **feat:** Untuk fitur baru (Contoh: **feat: add return book function**)[cite: 1].
- **fix:** Untuk perbaikan bug (Contoh: **fix: resolve JWT token expired**)[cite: 1].
- **docs:** Untuk dokumentasi (Contoh: **docs: update workflow guide**)[cite: 1].
- **chore:** Untuk *maintenance* (Contoh: **chore: setup docker compose**)[cite: 1].
- **style:** Untuk merapikan *indentation* atau UI tanpa ubah logika[cite: 1].

4. Proses Pull Request (PR) & Code Review

1. **Buat PR:** Setelah *push branch* Anda, segera buka PR di GitHub[cite: 1].
2. **Reviewer Otomatis:** Sistem akan secara otomatis menandai *reviewer* berdasarkan file **.github/CODEOWNERS** yang sudah disiapkan oleh Lead DevOps[cite: 1].
3. **Syarat Merge:** PR hanya bisa di-*merge* jika sudah mendapatkan minimal **1 Approve** dari *reviewer* yang ditugaskan[cite: 1].
4. **Metode Merge:** Gunakan metode **Squash and Merge** agar riwayat *commit* di *main* tetap bersih menjadi 1 *commit* per fitur[cite: 1].

